

Università degli Studi di Milano Bicocca
Scuola di Scienze
Dipartimento di Informatica, Sistemistica e Comunicazione

Metodi del Calcolo Scientifico

Progetto 1

Solutori iterativi per sistemi lineari simmetrici e definiti positivi

Relazione

Matias Bonoli

Anno Accademico 2025-2026

24 giugno 2026

Indice

1	Introduzione	2
1.1	Linguaggio e dipendenze	2
2	PARTE 1	2
2.1	I quattro metodi e il criterio di arresto	2
2.2	Metodi stazionari: lo splitting $A = P - N$	3
2.3	Metodi del gradiente: risolvere equivale a minimizzare	3
2.4	Un solo prodotto matrice–vettore per iterazione	3
3	PARTE 2	4
3.1	L’architettura della libreria	4
3.2	Note implementative rilevanti	4
3.3	Interfaccia da riga di comando	5
4	Esperimenti	5
4.1	Le matrici di test	5
4.2	Risultati numerici	5
4.3	Grafici di sintesi	8
4.4	Discussione comparativa	12
5	Documentazione del codice	13
5.1	Modulo <code>solvers.py</code>	13
5.2	Modulo <code>run_assignment.py</code>	13
5.3	Moduli <code>plot_results.py</code> e <code>analyze_cond.py</code>	14
5.4	Modulo <code>test_assignment.py</code>	14
A	Codice sorgente	14
A.1	<code>solvers.py</code>	14
A.2	<code>run_assignment.py</code>	20
A.3	<code>plot_results.py</code>	24
A.4	<code>analyze_cond.py</code>	25
A.5	<code>test_assignment.py</code>	26

1 Introduzione

L'obiettivo principale di questo progetto è lo studio e l'implementazione pratica dei principali *metodi iterativi* per la risoluzione di sistemi lineari $A\mathbf{x} = \mathbf{b}$, nel caso in cui la matrice A sia simmetrica e definita positiva (SPD). Il lavoro che abbiamo svolto si articola in due fasi principali.

Nella prima parte abbiamo curato l'inquadramento algoritmico dei quattro metodi richiesti dalla traccia — Jacobi, Gauss–Seidel, Gradiente e Gradiente coniugato — partendo dalla loro derivazione teorica (lo *splitting* $A = P - N$ per i metodi stazionari e l'interpretazione come problema di minimo per i metodi del gradiente) e fissando il criterio di arresto e la procedura di validazione comune a tutti.

Nella seconda parte abbiamo sviluppato una piccola *libreria* che implementa concretamente questi algoritmi. Abbiamo scelto un'architettura coesa, in cui lo scheletro di iterazione è scritto una sola volta e ogni metodo fornisce soltanto la propria regola di aggiornamento, e abbiamo dotato il progetto di un eseguibile da riga di comando che, su un insieme di matrici reali, misura per ciascun metodo il numero di iterazioni, l'errore di ricostruzione e i tempi di calcolo al variare della tolleranza richiesta.

1.1 Linguaggio e dipendenze

Il progetto è interamente scritto in **Python 3.12**. Tutti i pacchetti utilizzati sono *open source*:

- **numpy**: gestione di array e operazioni vettoriali (prodotto scalare, norma euclidea, aggiornamenti elemento per elemento);
- **scipy.sparse**: struttura dati per la matrice in formato sparso CSR e prodotto matrice–vettore $A\mathbf{v}$;
- **scipy.io.mmread**: lettura dei file in formato MatrixMarket `.mtx`;
- **numba**: compilazione JIT della sola *sweep* di Gauss–Seidel (intrinsecamente sequenziale), per renderla veloce restando codice nostro;
- **matplotlib**: generazione dei grafici di sintesi;
- **scipy.sparse.linalg.eigsh**: usato *solo* nello script di analisi a margine, per stimare gli autovalori estremi e quindi $\text{cond}(A)$ ai fini dei commenti (non fa parte dei solutori).

È importante sottolineare che, come consentito esplicitamente dalla consegna, **scipy/numpy** sono usati **esclusivamente** per le strutture dati e le operazioni elementari: **gli algoritmi risolutivi sono interamente sviluppati nel progetto**. Non viene usato alcun solutore di sistemi lineari di libreria (niente `spsolve`, `cg`, `numpy.linalg.solve` e simili).

2 PARTE 1

2.1 I quattro metodi e il criterio di arresto

Tutti i metodi che abbiamo implementato assumono A **SPD** ($A = A^T$ e $\mathbf{v}^T A \mathbf{v} > 0$ per ogni $\mathbf{v} \neq \mathbf{0}$) e partono dallo stesso vettore iniziale nullo $\mathbf{x}^{(0)} = \mathbf{0}$. Indichiamo con

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$$

il residuo alla generica iterazione k . Il criterio di arresto, comune a tutti i metodi e richiesto dalla traccia, è formulato sul **residuo scalato**:

$$\frac{\|\mathbf{b} - A\mathbf{x}^{(k)}\|}{\|\mathbf{b}\|} < \text{tol}.$$

A protezione dei casi di non convergenza, l'iterazione si arresta comunque al superamento di un numero massimo di passi `max_iter = 20000`, segnalando in tal caso `converged = False`.

2.2 Metodi stazionari: lo splitting $A = P - N$

L'idea alla base dei metodi stazionari è spezzare $A = P - N$ con P facile da invertire; il sistema diventa l'iterazione di punto fisso

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + P^{-1}\mathbf{r}^{(k)}.$$

Un principio centrale che governa la convergenza è il seguente: il metodo converge per ogni $\mathbf{x}^{(0)}$ se e solo se il raggio spettrale della *matrice di iterazione* $B = P^{-1}N$ soddisfa $\rho(B) < 1$. Quanto più piccolo è $\rho(B)$, tanto più rapidamente l'errore viene abbattuto.

- **Jacobi**: scegliamo $P = D$ (la diagonale di A). L'inversa D^{-1} è banale (i reciproci degli elementi diagonali), quindi il passo è completamente vettoriale e parallelizzabile. La sola ipotesi SPD *non* garantisce $\rho(B) < 1$ (una condizione sufficiente è che anche $2D - A$ sia SPD, oppure la dominanza diagonale di A).
- **Gauss–Seidel**: scegliamo $P = D + L$ (la parte triangolare inferiore di A). Riusando immediatamente le componenti appena aggiornate, di norma servono *circa la metà* delle iterazioni di Jacobi. Inoltre, se A è SPD, Gauss–Seidel converge **sempre**. Il prezzo è che il passo è una sostituzione in avanti, intrinsecamente sequenziale.

2.3 Metodi del gradiente: risolvere equivale a minimizzare

Per A SPD, risolvere $A\mathbf{x} = \mathbf{b}$ equivale a minimizzare il funzionale quadratico

$$\varphi(\mathbf{y}) = \frac{1}{2} \mathbf{y}^T A \mathbf{y} - \mathbf{b}^T \mathbf{y}, \quad \nabla \varphi(\mathbf{y}) = A \mathbf{y} - \mathbf{b},$$

il cui gradiente si annulla esattamente in $A\mathbf{y} = \mathbf{b}$. Proprio perché A è definita positiva, φ è un paraboloide a “ciotola” e quel punto stazionario è un minimo.

- **Gradiente** (*steepest descent*): a ogni passo ci si muove nella direzione dell'antigradiente $-\nabla \varphi = \mathbf{r}^{(k)}$, con il passo ottimo lungo quella retta

$$\alpha_k = \frac{\mathbf{r}^{(k)} \cdot \mathbf{r}^{(k)}}{\mathbf{r}^{(k)} \cdot A \mathbf{r}^{(k)}}, \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}.$$

Il metodo converge sempre per A SPD, ma sulle matrici mal condizionate la direzione di massima discesa non punta verso il fondo della valle: si genera il caratteristico *zig-zag* e la velocità degrada con il numero di condizionamento $\text{cond}(A) = \lambda_{\max}/\lambda_{\min}$.

- **Gradiente coniugato** (CG): le direzioni di ricerca sono A -coniugate ($\mathbf{d}_i^T A \mathbf{d}_j = 0$ per $i \neq j$). Una volta ottimizzata una direzione, le successive non la “rovinano”: niente zig-zag. Per A SPD di dimensione n , CG raggiunge la soluzione esatta in **al più n iterazioni**; in pratica ne bastano molte meno e il numero di iterazioni scala come $\sqrt{\text{cond}(A)}$.

2.4 Un solo prodotto matrice–vettore per iterazione

Una scelta implementativa che attraversa tutta la PARTE 2 ma che ha radice algoritmica è quella di usare **un solo** prodotto matrice–vettore per iterazione. Per Gradiente e Gradiente coniugato il residuo successivo non viene ricalcolato come $\mathbf{b} - A\mathbf{x}^{(k+1)}$ (un secondo matvec), ma ottenuto per ricorrenza

$$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k A \mathbf{v},$$

riusando il prodotto $A\mathbf{v}$ già calcolato per determinare il passo. Su matrici dense questo dimezza il costo dominante dell'iterazione.

3 PARTE 2

3.1 L'architettura della libreria

La libreria che abbiamo sviluppato (file `solvers.py`) **non è una sequenza di funzioni indipendenti**, ma una piccola gerarchia di classi coesa, costruita su un'osservazione: tutti i metodi condividono lo stesso scheletro e differiscono *soltanto* nel passo di aggiornamento $\mathbf{x}^{(k)} \rightarrow \mathbf{x}^{(k+1)}$. L'architettura si articola in questi componenti fondamentali:

1. **Lo scheletro comune.** La classe base astratta `IterativeSolver` implementa *una sola volta* il metodo `solve()`: inizializza $\mathbf{x}^{(0)} = \mathbf{0}$, esegue il ciclo che si arresta sul residuo scalato (o al superamento di `max_iter`), misura il tempo e costruisce il risultato.

```
x^(0) = 0
finche' ||b - A x^(k)|| / ||b|| >= tol e k < max_iter:
    x^(k+1) = aggiornamento(x^(k))
    k += 1
```

2. **Il passo specifico.** L'interfaccia astratta `_Iteration` incapsula lo stato e il *passo* di un metodo (pattern *template method*). Le quattro classi concrete — `Jacobi`, `GaussSeidel`, `Gradient`, `ConjugateGradient` — forniscono unicamente la propria regola di aggiornamento. Aggiungere un quinto metodo significa scrivere una sola classe, senza toccare lo scheletro.
3. **Il risultato uniforme.** La dataclass `SolverResult` raccoglie in modo omogeneo l'esito di ogni risoluzione: soluzione approssimata, iterazioni, tempo, residuo relativo finale e flag di convergenza.

Le istanze dei quattro metodi sono raccolte nella tupla `SOLVERS`, su cui iterano sia il runner sia i test. Le parti di servizio comuni sono fattorizzate una sola volta (`_validate_for_iteration` per controllare che la matrice sia quadrata e con diagonale non nulla; `_safe_norm` per il calcolo della norma di $\mathbf{b}$).

3.2 Note implementative rilevanti

L'implementazione degli algoritmi è strutturata attorno a quattro accorgimenti:

- **Sparsità preservata.** Nessun metodo fattorizza o modifica A : la matrice è impiegata solo per prodotti $A\mathbf{v}$, il cui costo è proporzionale al numero di non-zeri e non a n^2 (nessun *fill-in*).
- **Un solo matvec per iterazione.** Come descritto in PARTE 1, per Jacobi, Gradiente e Gradiente coniugato il residuo successivo si ottiene per ricorrenza, evitando un prodotto matrice-vettore aggiuntivo.
- **Gauss-Seidel senza inversa, ma veloce.** Il passo $P^{-1}\mathbf{r}$ (con $P = D + L$) è realizzato da una *sweep* in place che aggiorna $\mathbf{x}^{(k+1)}$ riusando i valori appena calcolati — equivalente a una sostituzione in avanti — **senza mai costruire** P^{-1} . Essendo intrinsecamente sequenziale (x_i dipende da $x_0, \dots, x_{i-1}$), questa sweep è compilata con **numba** (`_gauss_seidel_sweep`), restando codice nostro: così non diventa il collo di bottiglia che sarebbe un ciclo Python puro.
- **Tempi equi.** Prima delle misure eseguiamo un *warm-up* che forza la compilazione JIT su un problemino 3×3 , così il tempo riportato per Gauss-Seidel non include il costo (una tantum) di compilazione.

3.3 Interfaccia da riga di comando

Per applicare e mettere a confronto i metodi abbiamo realizzato un eseguibile (`run_assignment.py`). L'utente può lanciare l'intera validazione su tutte le matrici e tolleranze, oppure restringere il calcolo a una specifica matrice (`--matrix`), tolleranza (`--tol`) o metodo (`--method`), parametri tutti ripetibili. Il software produce un CSV numerico con tutte le metriche e tre grafici di sintesi (iterazioni, errore relativo e tempo al variare della tolleranza), salvabili in una cartella di output configurabile.

4 Esperimenti

Per ogni matrice abbiamo seguito la procedura di validazione standard richiesta dalla traccia: fissata la soluzione esatta nota $\mathbf{x} = [1, 1, \dots, 1]$, il termine noto è $\mathbf{b} = A\mathbf{x}$ (cioè il vettore delle somme di riga di A), e i quattro metodi partono da $\mathbf{x}^{(0)} = \mathbf{0}$. Avere a disposizione la soluzione esatta ci permette di misurare l'errore vero

$$\frac{\|\mathbf{x}_{\text{calc}} - \mathbf{x}\|}{\|\mathbf{x}\|},$$

cosa impossibile in un problema reale in cui la soluzione non è nota. Ogni configurazione è stata ripetuta alle quattro tolleranze $\text{tol} \in \{10^{-4}, 10^{-6}, 10^{-8}, 10^{-10}\}$, con `max_iter` = 20000.

Le misure di tempo sono state condotte sempre sullo stesso ambiente: Intel Core i7-1165G7 @ 2.80 GHz, 16 GB RAM, Windows 11, Python 3.12 con numpy, scipy e numba.

4.1 Le matrici di test

Le quattro matrici fornite si dividono in due famiglie con caratteristiche opposte, come riassunto nella Tabella 1.

Tabella 1: Caratteristiche delle matrici di test (autovalori e $\text{cond}(A)$ stimati con `analyze_cond.py`).

Matrice	n	non-zeri	$\lambda_{\min}$	$\lambda_{\max}$	$\text{cond}(A)$
spa1	1000	182 434	$4.88 \cdot 10^{-1}$	$9.99 \cdot 10^2$	$2.05 \cdot 10^3$
spa2	3000	1 633 298	$2.12 \cdot 10^0$	$3.00 \cdot 10^3$	$1.41 \cdot 10^3$
vem1	1681	13 385	$1.23 \cdot 10^{-2}$	$4.00 \cdot 10^0$	$3.25 \cdot 10^2$
vem2	2601	21 225	$7.89 \cdot 10^{-3}$	$4.00 \cdot 10^0$	$5.07 \cdot 10^2$

Le **spa*** sono molto più dense (≈ 180 e ≈ 540 non-zeri per riga) e con condizionamento maggiore; le **vem*** sono molto più sparse (≈ 8 non-zeri per riga) e meglio condizionate. Come vedremo, questa differenza strutturale si riflette in modo netto sulle prestazioni dei diversi metodi.

4.2 Risultati numerici

Le tabelle seguenti riportano i risultati completi estratti da `results/results.csv`. Tutti i metodi sono arrivati a convergenza (`conv` = sì) entro `max_iter` = 20000. I tempi sono in secondi sull'ambiente indicato (soggetti a variazioni di runtime fra esecuzioni).

Tabella 2: **spa1** — $n = 1000$, $\text{nnz} = 182\,434$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	115	$1.77 \cdot 10^{-3}$	$9.48 \cdot 10^{-5}$	0.080
10^{-4}	Gauss–Seidel	9	$1.82 \cdot 10^{-2}$	$7.79 \cdot 10^{-5}$	0.017
10^{-4}	Gradiente	143	$3.46 \cdot 10^{-2}$	$9.91 \cdot 10^{-5}$	0.093
10^{-4}	Gradiente coniugato	49	$2.08 \cdot 10^{-2}$	$9.78 \cdot 10^{-5}$	0.031
10^{-6}	Jacobi	181	$1.80 \cdot 10^{-5}$	$9.62 \cdot 10^{-7}$	0.110
10^{-6}	Gauss–Seidel	17	$1.30 \cdot 10^{-4}$	$5.58 \cdot 10^{-7}$	0.031
10^{-6}	Gradiente	3577	$9.68 \cdot 10^{-4}$	$9.98 \cdot 10^{-7}$	2.172
10^{-6}	Gradiente coniugato	134	$2.55 \cdot 10^{-5}$	$9.74 \cdot 10^{-7}$	0.091
10^{-8}	Jacobi	247	$1.82 \cdot 10^{-7}$	$9.76 \cdot 10^{-9}$	0.168
10^{-8}	Gauss–Seidel	24	$1.71 \cdot 10^{-6}$	$7.34 \cdot 10^{-9}$	0.038
10^{-8}	Gradiente	8233	$9.82 \cdot 10^{-6}$	$9.99 \cdot 10^{-9}$	5.819
10^{-8}	Gradiente coniugato	177	$1.32 \cdot 10^{-7}$	$9.03 \cdot 10^{-9}$	0.140
10^{-10}	Jacobi	313	$1.85 \cdot 10^{-9}$	$9.91 \cdot 10^{-11}$	0.241
10^{-10}	Gauss–Seidel	31	$2.25 \cdot 10^{-8}$	$9.65 \cdot 10^{-11}$	0.058
10^{-10}	Gradiente	12919	$9.82 \cdot 10^{-8}$	$9.99 \cdot 10^{-11}$	10.171
10^{-10}	Gradiente coniugato	200	$1.20 \cdot 10^{-9}$	$7.94 \cdot 10^{-11}$	0.156

Tabella 3: **spa2** — $n = 3000$, $\text{nnz} = 1\,633\,298$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	36	$1.77 \cdot 10^{-3}$	$9.54 \cdot 10^{-5}$	0.259
10^{-4}	Gauss–Seidel	5	$2.60 \cdot 10^{-3}$	$4.14 \cdot 10^{-5}$	0.088
10^{-4}	Gradiente	161	$1.81 \cdot 10^{-2}$	$9.90 \cdot 10^{-5}$	1.176
10^{-4}	Gradiente coniugato	42	$9.82 \cdot 10^{-3}$	$9.91 \cdot 10^{-5}$	0.306
10^{-6}	Jacobi	57	$1.67 \cdot 10^{-5}$	$9.00 \cdot 10^{-7}$	0.419
10^{-6}	Gauss–Seidel	8	$5.14 \cdot 10^{-5}$	$7.85 \cdot 10^{-7}$	0.146
10^{-6}	Gradiente	1949	$6.69 \cdot 10^{-4}$	$9.99 \cdot 10^{-7}$	12.249
10^{-6}	Gradiente coniugato	122	$1.20 \cdot 10^{-4}$	$9.09 \cdot 10^{-7}$	0.692
10^{-8}	Jacobi	78	$1.57 \cdot 10^{-7}$	$8.49 \cdot 10^{-9}$	0.480
10^{-8}	Gauss–Seidel	12	$2.79 \cdot 10^{-7}$	$4.27 \cdot 10^{-9}$	0.178
10^{-8}	Gradiente	5087	$6.87 \cdot 10^{-6}$	$9.98 \cdot 10^{-9}$	34.083
10^{-8}	Gradiente coniugato	196	$5.59 \cdot 10^{-7}$	$9.60 \cdot 10^{-9}$	1.387
10^{-10}	Jacobi	99	$1.48 \cdot 10^{-9}$	$8.01 \cdot 10^{-11}$	0.706
10^{-10}	Gauss–Seidel	15	$5.57 \cdot 10^{-9}$	$8.52 \cdot 10^{-11}$	0.262
10^{-10}	Gradiente	8285	$6.94 \cdot 10^{-8}$	$9.99 \cdot 10^{-11}$	58.376
10^{-10}	Gradiente coniugato	240	$5.32 \cdot 10^{-9}$	$9.82 \cdot 10^{-11}$	1.613

Tabella 4: **vem1** — $n = 1681$, $\text{nnz} = 13\,385$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	1314	$3.54 \cdot 10^{-3}$	$9.99 \cdot 10^{-5}$	0.084
10^{-4}	Gauss–Seidel	659	$3.51 \cdot 10^{-3}$	$9.93 \cdot 10^{-5}$	0.136
10^{-4}	Gradiente	890	$2.70 \cdot 10^{-3}$	$9.93 \cdot 10^{-5}$	0.098
10^{-4}	Gradiente coniugato	38	$4.08 \cdot 10^{-5}$	$7.11 \cdot 10^{-5}$	0.003
10^{-6}	Jacobi	2433	$3.54 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.219
10^{-6}	Gauss–Seidel	1218	$3.53 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.197
10^{-6}	Gradiente	1612	$2.71 \cdot 10^{-5}$	$9.97 \cdot 10^{-7}$	0.155
10^{-6}	Gradiente coniugato	45	$3.73 \cdot 10^{-7}$	$8.90 \cdot 10^{-7}$	0.004
10^{-8}	Jacobi	3552	$3.54 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.273
10^{-8}	Gauss–Seidel	1778	$3.52 \cdot 10^{-7}$	$9.96 \cdot 10^{-9}$	0.328
10^{-8}	Gradiente	2336	$2.70 \cdot 10^{-7}$	$9.90 \cdot 10^{-9}$	0.231
10^{-8}	Gradiente coniugato	53	$2.83 \cdot 10^{-9}$	$7.80 \cdot 10^{-9}$	0.005
10^{-10}	Jacobi	4671	$3.54 \cdot 10^{-9}$	$9.99 \cdot 10^{-11}$	0.352
10^{-10}	Gauss–Seidel	2338	$3.51 \cdot 10^{-9}$	$9.94 \cdot 10^{-11}$	0.370
10^{-10}	Gradiente	3058	$2.71 \cdot 10^{-9}$	$9.97 \cdot 10^{-11}$	0.297
10^{-10}	Gradiente coniugato	59	$2.19 \cdot 10^{-11}$	$6.91 \cdot 10^{-11}$	0.005

Tabella 5: **vem2** — $n = 2601$, $\text{nnz} = 21\,225$.

tol	metodo	iter	err. rel.	res. scal.	t [s]
10^{-4}	Jacobi	1927	$4.97 \cdot 10^{-3}$	$9.99 \cdot 10^{-5}$	0.198
10^{-4}	Gauss–Seidel	965	$4.95 \cdot 10^{-3}$	$9.98 \cdot 10^{-5}$	0.223
10^{-4}	Gradiente	1308	$3.81 \cdot 10^{-3}$	$9.98 \cdot 10^{-5}$	0.159
10^{-4}	Gradiente coniugato	47	$5.73 \cdot 10^{-5}$	$9.19 \cdot 10^{-5}$	0.006
10^{-6}	Jacobi	3676	$4.97 \cdot 10^{-5}$	$9.99 \cdot 10^{-7}$	0.381
10^{-6}	Gauss–Seidel	1840	$4.94 \cdot 10^{-5}$	$9.96 \cdot 10^{-7}$	0.557
10^{-6}	Gradiente	2438	$3.79 \cdot 10^{-5}$	$9.92 \cdot 10^{-7}$	0.321
10^{-6}	Gradiente coniugato	56	$4.74 \cdot 10^{-7}$	$8.54 \cdot 10^{-7}$	0.009
10^{-8}	Jacobi	5425	$4.97 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.738
10^{-8}	Gauss–Seidel	2714	$4.96 \cdot 10^{-7}$	$9.99 \cdot 10^{-9}$	0.720
10^{-8}	Gradiente	3566	$3.81 \cdot 10^{-7}$	$9.97 \cdot 10^{-9}$	0.588
10^{-8}	Gradiente coniugato	66	$4.30 \cdot 10^{-9}$	$8.60 \cdot 10^{-9}$	0.011
10^{-10}	Jacobi	7174	$4.96 \cdot 10^{-9}$	$9.98 \cdot 10^{-11}$	0.819
10^{-10}	Gauss–Seidel	3589	$4.95 \cdot 10^{-9}$	$9.97 \cdot 10^{-11}$	0.230
10^{-10}	Gradiente	4696	$3.80 \cdot 10^{-9}$	$9.94 \cdot 10^{-11}$	0.157
10^{-10}	Gradiente coniugato	74	$2.25 \cdot 10^{-11}$	$5.90 \cdot 10^{-11}$	0.002

4.3 Grafici di sintesi

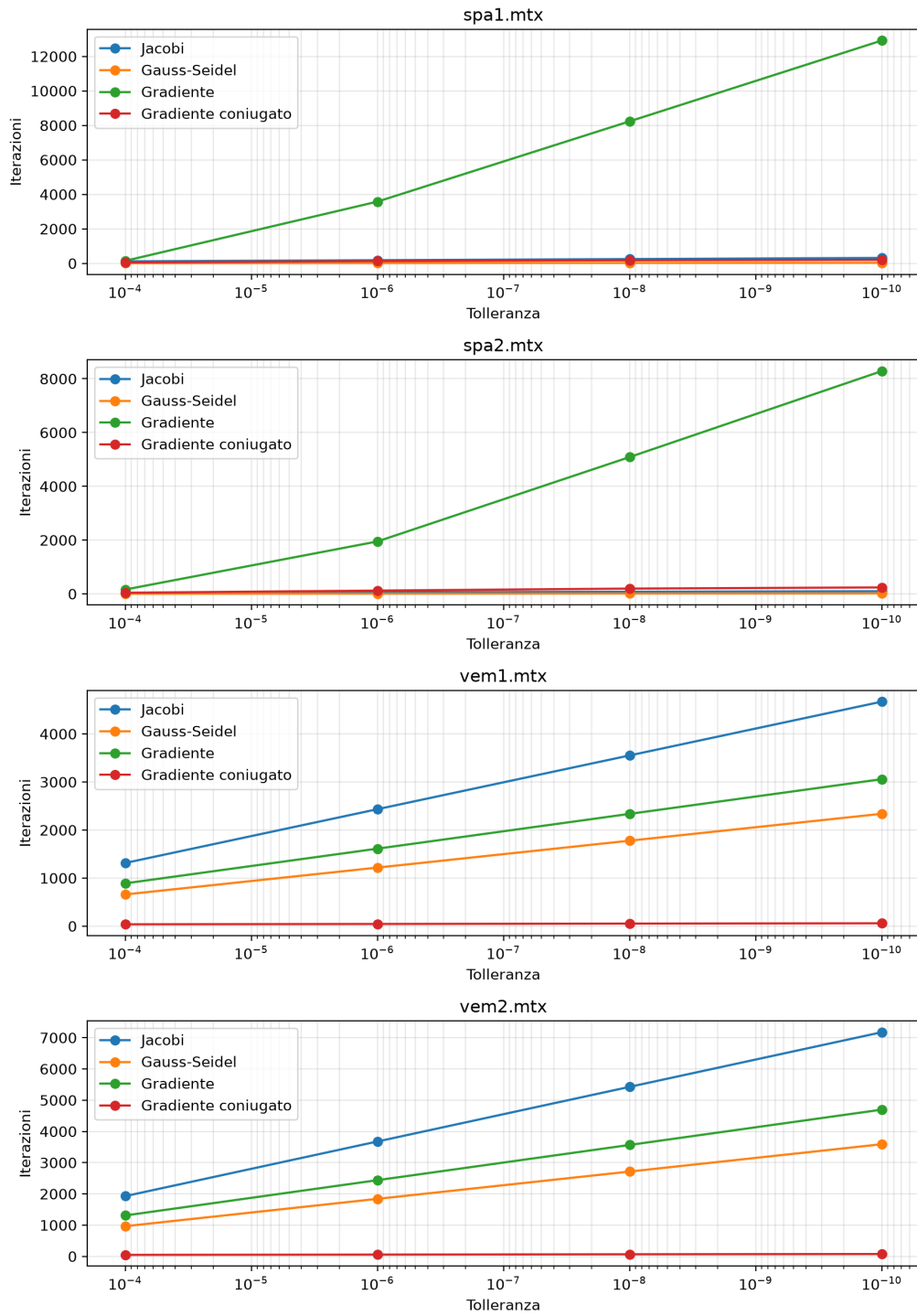


Figura 1: Numero di iterazioni al variare di tol (asse x in scala logaritmica, una riga per matrice).

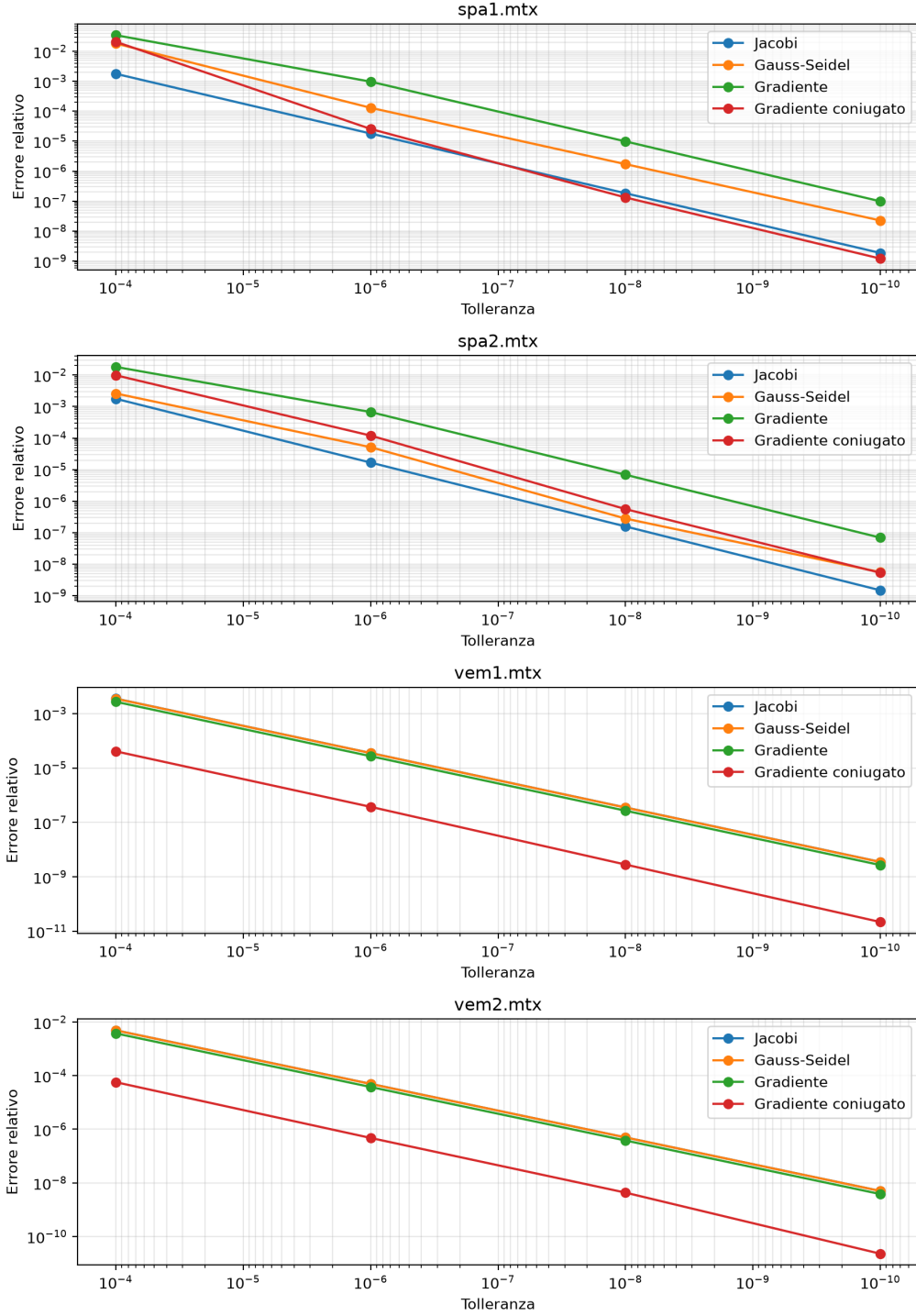


Figura 2: Errore relativo $\|\mathbf{x}_{\text{calc}} - \mathbf{x}\|/\|\mathbf{x}\|$ al variare di tol (scala logaritmica).

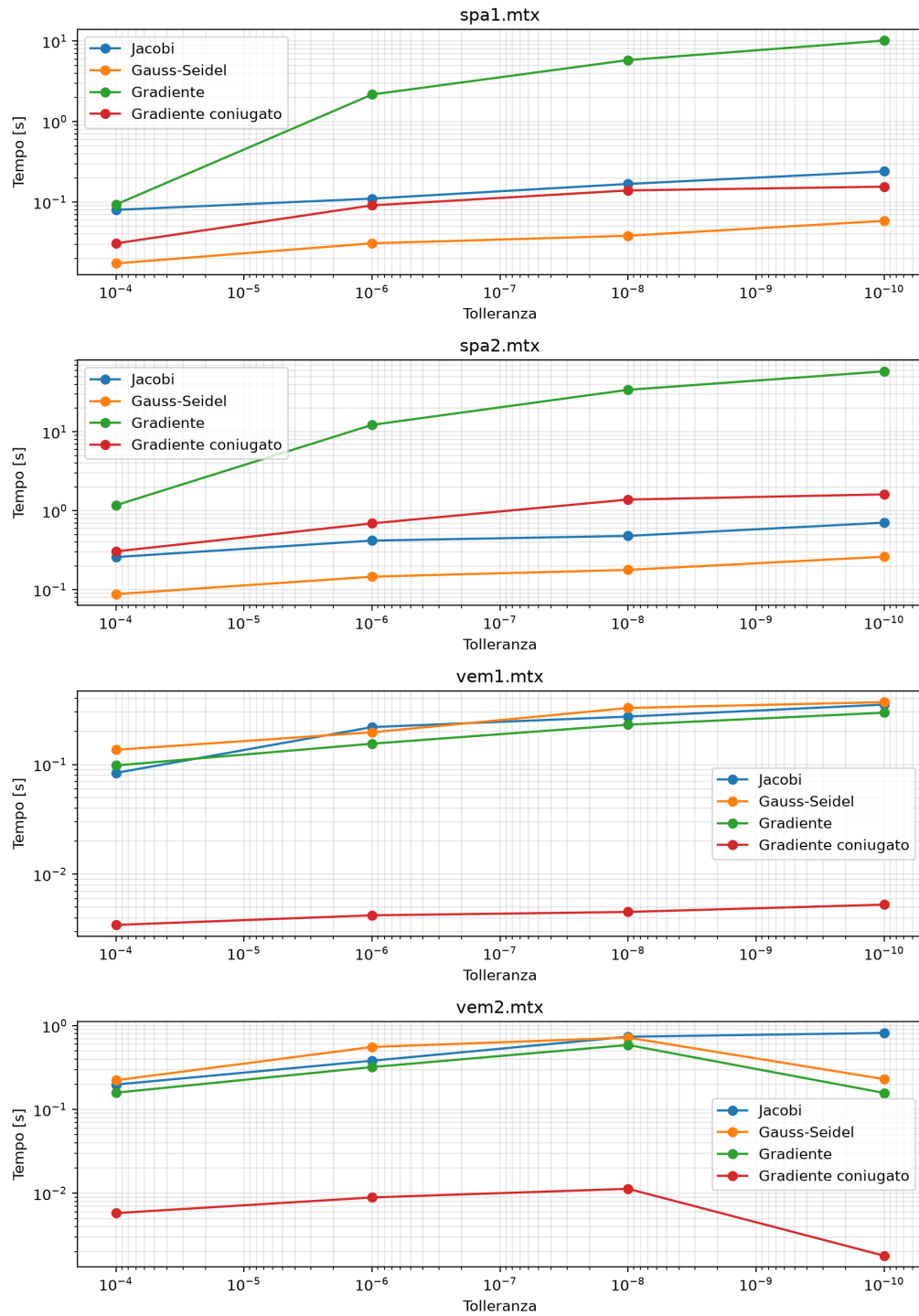


Figura 3: Tempo di calcolo al variare di tol (scala logaritmica).

4.4 Discussione comparativa

I dati numerici e i grafici restituiscono un quadro coerente con la teoria della PARTE 1. Riassumiamo le osservazioni principali.

Gauss–Seidel $\approx$ metà delle iterazioni di Jacobi. È esattamente quanto previsto: riusando immediatamente le componenti appena calcolate, Gauss–Seidel fa più “lavoro utile” per iterazione. Ad esempio su **vem2** a $\text{tol} = 10^{-8}$ Jacobi richiede 5425 iterazioni contro le 2714 di Gauss–Seidel (rapporto ≈ 2.0). Il singolo passo di Gauss–Seidel è però un po’ più costoso (sweep sequenziale), perciò sulle matrici molto sparse **vem** i tempi totali dei due metodi si equivalgono, mentre sulle **spa** (dove Gauss–Seidel converge in pochissime iterazioni) è nettamente il più rapido.

Due “regimi” opposti tra le famiglie di matrici. I metodi stazionari e quelli del gradiente rispondono a proprietà diverse della matrice:

- su **spa1/spa2** Jacobi e Gauss–Seidel sono fulminei (5–313 iterazioni), mentre il Gradiente è lentissimo (fino a $\sim 12\,900$ iterazioni). Le **spa** sono fortemente *dominanti diagonali* $\Rightarrow$ raggio spettrale della matrice di iterazione piccolo $\Rightarrow$ metodi stazionari rapidi; ma hanno $\text{cond}(A) \approx 2 \cdot 10^3$, e la velocità del Gradiente dipende proprio da $\text{cond}(A)$, da cui la lentezza;
- su **vem1/vem2** la situazione si riequilibra: la minore dominanza diagonale rende Jacobi/Gauss–Seidel più lenti (migliaia di iterazioni), mentre il minore $\text{cond}(A) \approx 3\text{--}5 \cdot 10^2$ rende il Gradiente competitivo.

Ciò conferma che la velocità di Jacobi/Gauss–Seidel è governata dalla **dominanza diagonale** ($\rho(P^{-1}N)$), mentre quella di Gradiente/Gradiente coniugato è governata dal **numero di condizionamento**.

Il Gradiente coniugato è il metodo più robusto e regolare. Converte sempre in **poche decine di iterazioni** (38–240), indipendentemente dalla matrice, con crescita lentissima al ridursi di tol . Il numero di iterazioni scala come $\sqrt{\text{cond}(A)}$ (coerentemente, **spa1** con $\text{cond} \approx 2 \cdot 10^3$ richiede più passi di **vem1** con $\text{cond} \approx 3 \cdot 10^2$) e resta sempre ben al di sotto del limite teorico di n iterazioni. È il vincitore netto sulle matrici mal condizionate **vem**.

Residuo piccolo $\neq$ errore piccolo. A parità di tol , l’errore relativo sulla soluzione **non** è uguale fra i metodi e, per quelli stazionari, vale circa $\text{tol} \times \text{cost}$, con cost legato al condizionamento (relazione $\text{errore} \lesssim \text{cond}(A) \cdot \text{residuo}$). Ad esempio a $\text{tol} = 10^{-6}$: su **vem2** l’errore è $\sim 5 \cdot 10^{-5}$ (circa $50\times$ il residuo); su **spa1** il Gradiente ha errore $\sim 10^{-3}$ (circa $1000\times$ il residuo). Il Gradiente coniugato raggiunge invece errori molto più piccoli (fino a $\sim 10^{-11}$), perché tende a ridurre simultaneamente tutte le componenti spettrali dell’errore.

Tempi di calcolo. In secondi assoluti il quadro dipende dalla densità della matrice:

- sulle **spa** (dense) il prodotto matrice–vettore è caro, quindi il Gradiente, che fa molte iterazioni, arriva a oltre 50 s (**spa2** a 10^{-10}), mentre Gauss–Seidel resta sotto 0.3 s e CG sotto 1.7 s;
- sulle **vem** (molto sparse) tutti i metodi sono rapidi (< 1 s); CG è comunque il più veloce in assoluto (≤ 0.011 s) grazie al bassissimo numero di iterazioni.

In sintesi, su queste matrici il **Gradiente coniugato è la scelta migliore in ogni scenario**; tra i metodi stazionari, Gauss–Seidel domina Jacobi; il Gradiente semplice è penalizzato quando la matrice è mal condizionata (effetto “zig-zag”).

5 Documentazione del codice

Il codice è documentato in modo sistematico: ogni modulo ha una docstring di intestazione che ne descrive lo scopo, e ogni classe o funzione pubblica ha una docstring che ne specifica comportamento, argomenti, valore di ritorno ed eccezioni sollevate. Di seguito si riporta la documentazione modulo per modulo.

5.1 Modulo `solvers.py`

È il cuore della libreria: contiene i quattro metodi e lo scheletro comune.

SolverResult (dataclass). Raccoglie l'esito di una risoluzione: `method` (nome del metodo), `x` (soluzione approssimata), `iterations`, `elapsed` (tempo in secondi), `residual_relative` (residuo scalato finale), `converged`.

_Iteration (interfaccia astratta). Incapsula lo stato di un metodo per un singolo problema. Espone due metodi astratti: `residual_norm()` (norma euclidea del residuo corrente) e `step()` (aggiornamento $\mathbf{x}^{(k)} \rightarrow \mathbf{x}^{(k+1)}$ in place).

IterativeSolver (classe base astratta). Implementa lo scheletro comune.

- `solve(A, b, tol, max_iter=None)`: risolve $A\mathbf{x} = \mathbf{b}$ da $\mathbf{x}^{(0)} = \mathbf{0}$, arrestandosi sul residuo scalato o al superamento di `max_iter`. *Output*: **SolverResult**. *Eccezioni*: **ValueError** se la matrice non è quadrata o ha diagonale nulla.
- `_make_iteration(A, b, x)`: metodo astratto che costruisce l'oggetto iterazione specifico del metodo.

Jacobi / GaussSeidel / Gradient / ConjugateGradient. Le quattro classi concrete; ciascuna fornisce solo la propria regola di aggiornamento tramite la rispettiva classe `_Iteration` ($P = D$ per Jacobi; $P = D + L$ con sweep per Gauss-Seidel; passo ottimo lungo $\mathbf{r}$ per il Gradiente; direzioni A -coniugate per CG).

`_gauss_seidel_sweep(indptr, indices, data, diag, b, x)` (compilata con `@njit`). Esegue una sweep di Gauss-Seidel aggiornando `x` in place (sostituzione in avanti sul formato CSR).

`warmup()`. Forza la compilazione JIT su un problema 3×3 , da chiamare prima delle misure di tempo.

`_validate_for_iteration(A)` / `_safe_norm(x)`. Funzioni di servizio: la prima controlla che A sia quadrata e con diagonale non nulla; la seconda restituisce la norma di un vettore, evitando la divisione per zero quando $\mathbf{b} = \mathbf{0}$.

5.2 Modulo `run_assignment.py`

Eseguibile da riga di comando per la validazione.

- `main()`: parse gli argomenti, esegue il `warmup()`, e per ogni matrice e tolleranza risolve con i metodi selezionati, calcola l'errore relativo e accumula le righe del CSV; salvo `--no-plots` genera anche i grafici.
- `parse_args()`: definisce la CLI (`--matrix`, `--tol`, `--method`, `--max-iter`, `--output`, `--no-plots`, i primi tre ripetibili).
- `_resolve_matrices` / `_select_solvers`: risolvono rispettivamente l'elenco dei file `.mtx` (tutta la cartella `dati` se non specificato) e il sottoinsieme di metodi da usare.
- `_write_csv(rows, path)`: serializza tutte le metriche (matrice, n , nnz, tol, metodo, errore relativo, residuo, iterazioni, tempo, convergenza) in `results/results.csv`.

5.3 Moduli `plot_results.py` e `analyze_cond.py`

`plot_results.py`. `create_plots(rows, output_dir)` produce i tre grafici (errore relativo, iterazioni, tempo); la funzione ausiliaria `_plot_metric` disegna una metrica raggruppando per matrice e metodo, con l'asse `tol` in scala logaritmica e invertito.

`analyze_cond.py`. Script a margine (non parte dei solutori): stima $\lambda_{\min}$, $\lambda_{\max}$ e $\text{cond}(A)$ per ogni matrice tramite `scipy.sparse.linalg.eigsh`, ai soli fini dei commenti della relazione.

5.4 Modulo `test_assignment.py`

Test automatici: `test_scipy_reads_matrix_market` (parser MatrixMarket), `test_solvers_on_tridiagonal` (convergenza dei quattro metodi su una tridiagonale SPD) e `test_conjugate_gradient_is_fast_on_small_matrices` (CG converge in al più n iterazioni su un sistema piccolo).

A Codice sorgente

Di seguito si riporta il codice sorgente completo dei moduli del progetto. La documentazione modulo per modulo è descritta nella Sezione 5.

A.1 `solvers.py`

```
1  """Solutori iterativi per sistemi lineari Ax = b con A simmetrica
   definita positiva.
2
3  ARCHITETTURA
4  -----
5  La libreria NON e' una sequenza di funzioni indipendenti, ma una
   piccola
6  gerarchia di classi coese, costruita su un'osservazione delle
   dispense: tutti i
7  metodi condividono lo stesso scheletro (Algoritmo 6) e differiscono
   SOLO nel
8  passo di aggiornamento x^(k) -> x^(k+1).
9
10     IterativeSolver (classe base astratta)
11     |-- implementa UNA volta sola lo scheletro comune in solve():
12     |     x^(0)=0; ciclo con criterio di arresto sul residuo
       scalato;
13     |     controllo su max_iter; misura del tempo; costruzione
       del risultato.
14     |
15     |-- Jacobi
16     |-- GaussSeidel
17     |-- Gradient
18     |-- ConjugateGradient
19     ognuna fornisce solo la propria regola di aggiornamento
       ,
20     incapsulata in un oggetto _Iteration (pattern "template
       method").
21
22 Per le STRUTTURE DATI (matrice sparsa CSR, vettori) e le OPERAZIONI
   ELEMENTARI
23 (A@x, prodotto scalare, norma) ci si appoggia a scipy/numpy, come
   consentito
24 dalla consegna. Gli ALGORITMI risolutivi sono implementati
   interamente qui:
```

```

25 non si usa alcun solutore di sistemi lineari di scipy/numpy.
26
27 Criterio di arresto: residuo scalato  $\|b - A x^{(k)}\| / \|b\| < tol$ .
28 Controllo di sicurezza: arresto con converged=False se si supera
    max_iter.
29 """
30
31 from __future__ import annotations
32
33 from abc import ABC, abstractmethod
34 from dataclasses import dataclass
35 from time import perf_counter
36 from typing import Any
37
38 import numpy as np
39 from numba import njit
40
41
42 #
    -----
43 # Risultato di una risoluzione.
44 #
    -----
45 @dataclass
46 class SolverResult:
47     method: str          # nome del metodo
48     x: np.ndarray        # soluzione approssimata  $x^{(k)}$ 
49     iterations: int      # numero di iterazioni eseguite
50     elapsed: float       # tempo di calcolo in secondi
51     residual_relative: float #  $\|b - A x\| / \|b\|$  raggiunto
52     converged: bool      # True se il criterio di arresto e' stato
        soddisfatto
53
54
55 #
    -----
56 # Iterazione astratta + classe base con lo scheletro comune.
57 #
    -----
58 class _Iteration(ABC):
59     """Stato interno di un metodo iterativo per un singolo problema.
60
61     Mantiene (e aggiorna in place) il vettore soluzione 'x' e
        quanto serve per
62     calcolare il residuo ed eseguire il passo di aggiornamento.
63     """
64
65     @abstractmethod
66     def residual_norm(self) -> float:
67         """Norma euclidea del residuo corrente  $\|b - A x^{(k)}\|$ ."""
68
69     @abstractmethod
70     def step(self) -> None:
71         """Aggiorna lo stato:  $x^{(k)} \rightarrow x^{(k+1)}$ ."""

```



```

72
73
74 class IterativeSolver(ABC):
75     """Classe base: implementa lo scheletro comune a tutti i metodi."""
76
77     name: str = "iterative"
78     key: str = "iterative"           # identificativo per la CLI
79     DEFAULT_MAX_ITER: int = 20000
80
81     @abstractmethod
82     def _make_iteration(self, A: Any, b: np.ndarray, x: np.ndarray)
      -> _Iteration:
83         """Costruisce l'oggetto iterazione specifico del metodo."""
84
85     def solve(self, A: Any, b: np.ndarray, tol: float,
86               max_iter: int | None = None) -> SolverResult:
87         """Risolve  $Ax = b$  partendo da  $x^{(0)} = 0$ .
88
89         Si arresta quando il residuo scalato scende sotto ' $tol$ '
90         oppure quando
91         si superano ' $max\_iter$ ' iterazioni (in tal caso converged=
92         False).
93
94         """
95         _validate_for_iteration(A)
96         if max_iter is None:
97             max_iter = self.DEFAULT_MAX_ITER
98
99         b = np.asarray(b, dtype=float)
100         x = np.zeros_like(b, dtype=float)           # vettore iniziale
101             nullo
102         b_norm = _safe_norm(b)
103
104         start_time = perf_counter()
105         it = self._make_iteration(A, b, x)
106         iterations = 0
107         residual_relative = it.residual_norm() / b_norm   # all'
108             inizio vale 1
109         while residual_relative >= tol and iterations < max_iter:
110             it.step()
111             iterations += 1
112             residual_relative = it.residual_norm() / b_norm
113             elapsed = perf_counter() - start_time
114
115         return SolverResult(
116             method=self.name,
117             x=x.copy(),
118             iterations=iterations,
119             elapsed=elapsed,
120             residual_relative=residual_relative,
121             converged=residual_relative < tol,
122         )
123
124 #

```

```

121 # 1) Metodo di Jacobi.  $P = \text{diag}(A)$ ,  $x \leftarrow x + P^{-1} r$ ,  $r = b - A$ 
122 #  $x$ .
123 -----
124 class _JacobiIteration(_Iteration):
125     def __init__(self, A, b, x):
126         self.A = A
127         self.b = b
128         self.x = x
129         self.inv_diag = 1.0 / np.asarray(A.diagonal(), dtype=float)
130         #  $P^{-1}$ 
131         self.residual = b - A @ x # residuo iniziale
132         (= b)
133
134     def residual_norm(self):
135         return float(np.linalg.norm(self.residual))
136
137     def step(self):
138         #  $x^{(k+1)} = x^{(k)} + P^{-1} r^{(k)}$ 
139         self.x += self.inv_diag * self.residual
140         self.residual = self.b - self.A @ self.x
141
142 class Jacobi(IterativeSolver):
143     name = "Jacobi"
144     key = "jacobi"
145
146     def _make_iteration(self, A, b, x):
147         return _JacobiIteration(A, b, x)
148 # -----
149
150 # 2) Metodo di Gauss-Seidel.  $P$  = parte triangolare inferiore di  $A$ .
151 # sweep in place (= sostituzione in avanti), poi  $r = b - A x$ .
152 -----
153 class _GaussSeidelIteration(_Iteration):
154     def __init__(self, A, b, x):
155         self.A = A
156         self.b = b
157         self.x = x
158         self.diag = np.asarray(A.diagonal(), dtype=float)
159         self.indptr = A.indptr
160         self.indices = A.indices
161         self.data = np.asarray(A.data, dtype=float)
162         self.residual = b - A @ x
163
164     def residual_norm(self):
165         return float(np.linalg.norm(self.residual))
166
167     def step(self):
168         _gauss_seidel_sweep(self.indptr, self.indices, self.data,
169                             self.diag, self.b, self.x)
170         self.residual = self.b - self.A @ self.x

```

```

170
171
172 class GaussSeidel(IterativeSolver):
173     name = "Gauss-Seidel"
174     key = "gauss-seidel"
175
176     def _make_iteration(self, A, b, x):
177         return _GaussSeidelIteration(A, b, x)
178
179
180 #
-----
181 # 3) Metodo del Gradiente (steepest descent).
182 #      $\alpha = (r \cdot r) / (r \cdot A r)$  ;  $x \leftarrow x + \alpha r$  ;  $r \leftarrow r - \alpha A r$ .
183 #
-----
184 class _GradientIteration(_Iteration):
185     def __init__(self, A, b, x):
186         self.A = A
187         self.b = b
188         self.x = x
189         self.residual = b - A @ x # = -grad(phi) in
                                   x
190
191     def residual_norm(self):
192         return float(np.linalg.norm(self.residual))
193
194     def step(self):
195         Ar = self.A @ self.residual
196         denom = float(np.dot(self.residual, Ar))
197         alpha = float(np.dot(self.residual, self.residual)) / denom
198         self.x += alpha * self.residual
199         #  $r^{(k+1)} = b - A x^{(k+1)} = r^{(k)} - \alpha A r^{(k)}$ : niente
           matvec extra
200         self.residual = self.residual - alpha * Ar
201
202
203 class Gradient(IterativeSolver):
204     name = "Gradiente"
205     key = "gradient"
206
207     def _make_iteration(self, A, b, x):
208         return _GradientIteration(A, b, x)
209
210
211 #
-----
212 # 4) Metodo del Gradiente coniugato.
213 #     Direzioni A-coniugate: niente "zig-zag", convergenza in  $\leq n$ 
           passi.
214 #
-----
215 class _ConjugateGradientIteration(_Iteration):
216     def __init__(self, A, b, x):

```

```

217         self.A = A
218         self.b = b
219         self.x = x
220         self.residual = b - A @ x                                # residuo
                               iniziale (= b)
221         self.direction = self.residual.copy()                    #  $d^{(0)} = r^{(0)}$ 
222         self.residual_norm_sq = float(np.dot(self.residual, self.
            residual))
223
224     def residual_norm(self):
225         return float(np.sqrt(self.residual_norm_sq))
226
227     def step(self):
228         Ad = self.A @ self.direction
229         denom = float(np.dot(self.direction, Ad))
230         alpha = self.residual_norm_sq / denom
231         self.x += alpha * self.direction
232         self.residual = self.residual - alpha * Ad
233         new_norm_sq = float(np.dot(self.residual, self.residual))
234         beta = new_norm_sq / self.residual_norm_sq
235         self.direction = self.residual + beta * self.direction
236         self.residual_norm_sq = new_norm_sq
237
238
239 class ConjugateGradient(IterativeSolver):
240     name = "Gradiente coniugato"
241     key = "conjugate-gradient"
242
243     def _make_iteration(self, A, b, x):
244         return _ConjugateGradientIteration(A, b, x)
245
246
247 # Istanze dei quattro metodi, nell'ordine richiesto dalla consegna.
248 SOLVERS: tuple[IterativeSolver, ...] = (
249     Jacobi(),
250     GaussSeidel(),
251     Gradient(),
252     ConjugateGradient(),
253 )
254
255
256 #
257 -----
258
259 # Routine compilata (numba) e funzioni di servizio condivise.
260 #
261 -----
262
263 @jit(cache=True)
264 def _gauss_seidel_sweep(indptr, indices, data, diag, b, x):
265     """Una iterazione (sweep) di Gauss-Seidel, aggiornando x in place
266     .
267
268     Equivale a risolvere  $(D + L)y = r$  per sostituzione in avanti:
269     ogni  $x[i]$  usa
270     i valori già aggiornati  $x[0..i-1]$  e quelli vecchi  $x[i+1..n-1]$ . E
271     ,

```

```

265     un'operazione sequenziale (x[i] dipende dai precedenti) e quindi
        non
266     vettorizzabile: la compiliamo con numba per renderla veloce
        restando codice
267     nostro (non usiamo nessun solutore di libreria).
268     """
269     n = x.shape[0]
270     for i in range(n):
271         sigma = 0.0
272         for k in range(indptr[i], indptr[i + 1]):
273             j = indices[k]
274             if j != i:
275                 sigma += data[k] * x[j]
276             x[i] = (b[i] - sigma) / diag[i]
277
278
279 def warmup() -> None:
280     """Forza la compilazione JIT (numba) su un problemino 3x3.
281
282     Va chiamata PRIMA delle misure di tempo: altrimenti la prima
        esecuzione di
283     Gauss-Seidel paga il costo (una tantum) di compilazione della
        sweep,
284     falsando il tempo riportato.
285     """
286     from scipy.sparse import csr_matrix
287
288     A = csr_matrix(np.array([[4.0, 1.0, 0.0],
289                             [1.0, 4.0, 1.0],
290                             [0.0, 1.0, 4.0]]))
291     b = A @ np.ones(3)
292     for solver in SOLVERS:
293         solver.solve(A, b, 1e-12, 100)
294
295
296 def _validate_for_iteration(A: Any) -> None:
297     if A.shape[0] != A.shape[1]:
298         raise ValueError("La matrice deve essere quadrata")
299     diag = np.asarray(A.diagonal(), dtype=float)
300     if np.any(diag == 0):
301         raise ValueError("La matrice ha almeno un elemento diagonale
        nullo")
302
303
304 def _safe_norm(x: np.ndarray) -> float:
305     norm = float(np.linalg.norm(x))
306     if norm == 0:
307         return 1.0
308     return norm

```

A.2 run_assignment.py

```

1  """Command line runner for the first MCS assignment."""
2
3  from __future__ import annotations
4
5  import argparse

```

```

6 import csv
7 from pathlib import Path
8
9 import numpy as np
10 from scipy.io import mmread
11
12 from plot_results import create_plots
13 from solvers import SOLVERS, warmup
14
15
16 DEFAULT_TOLS = (1e-4, 1e-6, 1e-8, 1e-10)
17 DEFAULT_MAX_ITER = 20000
18 METHOD_NAMES = tuple(solver.key for solver in SOLVERS)
19
20
21 def main() -> None:
22     args = parse_args()
23     base_dir = Path(__file__).resolve().parent
24     output_dir = args.output
25     if not output_dir.is_absolute():
26         output_dir = base_dir / output_dir
27     output_dir.mkdir(parents=True, exist_ok=True)
28
29     matrix_paths = _resolve_matrices(args, base_dir)
30     tolerances = args.tol if args.tol else list(DEFAULT_TOLS)
31     solvers = _select_solvers(args.method)
32
33     warmup() # compilazione JIT (numba) prima delle misure di tempo
34
35     rows: list[dict[str, str]] = []
36     for matrix_path in matrix_paths:
37         print(f"\nMatrice: {matrix_path}")
38         A = mmread(matrix_path).tocsr().astype(float)
39         x_exact = np.ones(A.shape[0], dtype=float)
40         b = A @ x_exact
41
42         for tol in tolerances:
43             print(f"    tol = {tol:g}")
44             for solver in solvers:
45                 result = solver.solve(A, b, tol, args.max_iter)
46                 relative_error = np.linalg.norm(x_exact - result.x) /
47                     np.linalg.norm(x_exact)
48                 row = {
49                     "matrix": matrix_path.name,
50                     "n": str(A.shape[0]),
51                     "nnz": str(A.nnz),
52                     "tol": f"{tol:.0e}",
53                     "method": result.method,
54                     "relative_error": f"{relative_error:.16e}",
55                     "residual_relative": f"{result.residual_relative:.16e}",
56                     "iterations": str(result.iterations),
57                     "elapsed_seconds": f"{result.elapsed:.6f}",
58                     "converged": str(result.converged),
59                 }
60                 rows.append(row)
61                 print(

```

```

62         f"{result.method:20s} "
63         f"it={result.iterations:6d} "
64         f"err={relative_error:.3e} "
65         f"res={result.residual_relative:.3e} "
66         f"time={result.elapsed:.3f}s "
67         f"conv={result.converged}"
68     )
69
70     csv_path = output_dir / "results.csv"
71     _write_csv(rows, csv_path)
72     print(f"\nCSV scritto in: {csv_path}")
73
74     if args.plots:
75         plot_paths = create_plots(rows, output_dir)
76         print(f"Grafici scritti in: {output_dir / 'plots'}")
77         for plot_path in plot_paths:
78             print(f"    - {plot_path.name}")
79
80
81 def parse_args() -> argparse.Namespace:
82     parser = argparse.ArgumentParser(description="Mini libreria per
83         sistemi lineari SPD")
84     parser.add_argument(
85         "--matrix",
86         action="append",
87         type=Path,
88         help="Matrice .mtx da usare. Ripetibile. Se assente usa tutti
89             i file .mtx in dati.",
90     )
91     parser.add_argument(
92         "--tol",
93         action="append",
94         type=float,
95         help="Tolleranza da usare. Ripetibile. Se assente usa 1e-4, 1
96             e-6, 1e-8, 1e-10.",
97     )
98     parser.add_argument(
99         "--max-iter",
100         type=int,
101         default=DEFAULT_MAX_ITER,
102         help="Numero massimo di iterazioni, default 20000.",
103     )
104     parser.add_argument(
105         "--method",
106         action="append",
107         choices=METHOD_NAMES,
108         help=(
109             "Metodo da usare. Ripetibile. Se assente usa tutti i
110             metodi. "
111             f"Valori: {'', '}.join(METHOD_NAMES)}."
112         ),
113     )
114     parser.add_argument(
115         "--output",
116         type=Path,
117         default=Path("results"),
118         help="Cartella di output, default assignment 1/results.",
119     )

```

```

116     parser.add_argument(
117         "--no-plots",
118         dest="plots",
119         action="store_false",
120         help="Non generare grafici.",
121     )
122     parser.set_defaults(plots=True)
123     return parser.parse_args()
124
125
126 def _resolve_matrices(args: argparse.Namespace, base_dir: Path) ->
127     list[Path]:
128     if args.matrix:
129         paths = args.matrix
130     else:
131         data_dir = base_dir / "dati"
132         paths = sorted(data_dir.glob("*.mtx"))
133         if not paths:
134             raise FileNotFoundError(f"Nessuna matrice .mtx trovata in
135                                     : {data_dir}")
136
137     resolved = []
138     for path in paths:
139         if not path.is_absolute():
140             path = base_dir / path
141         if not path.exists():
142             raise FileNotFoundError(f"Matrice non trovata: {path}")
143         resolved.append(path)
144     return resolved
145
146
147 def _select_solvers(methods: list[str] | None):
148     if not methods:
149         return SOLVERS
150
151     selected = set(methods)
152     return tuple(solver for solver in SOLVERS if solver.key in
153                 selected)
154
155
156 def _write_csv(rows: list[dict[str, str]], csv_path: Path) -> None:
157     fieldnames = [
158         "matrix",
159         "n",
160         "nnz",
161         "tol",
162         "method",
163         "relative_error",
164         "residual_relative",
165         "iterations",
166         "elapsed_seconds",
167         "converged",
168     ]
169     with csv_path.open("w", encoding="utf-8", newline="") as file:
170         writer = csv.DictWriter(file, fieldnames=fieldnames)
171         writer.writeheader()
172         writer.writerows(rows)

```



```

171
172 if __name__ == "__main__":
173     main()

```

A.3 plot_results.py

```

1  """Plot utilities for assignment results."""
2
3  from __future__ import annotations
4
5  from collections import defaultdict
6  from pathlib import Path
7
8
9  def create_plots(rows: list[dict[str, str]], output_dir: str | Path)
   -> list[Path]:
10     output_dir = Path(output_dir)
11     plots_dir = output_dir / "plots"
12     plots_dir.mkdir(parents=True, exist_ok=True)
13
14     paths = [
15         plots_dir / "relative_error.png",
16         plots_dir / "iterations.png",
17         plots_dir / "time.png",
18     ]
19     _plot_metric(rows, "relative_error", "Errore relativo", paths[0],
20                 log_y=True)
21     _plot_metric(rows, "iterations", "Iterazioni", paths[1], log_y=
22                 False)
23     _plot_metric(rows, "elapsed_seconds", "Tempo [s]", paths[2],
24                 log_y=True)
25     return paths
26
27
28 def _plot_metric(
29     rows: list[dict[str, str]],
30     metric: str,
31     ylabel: str,
32     path: Path,
33     log_y: bool,
34 ) -> None:
35     import matplotlib.pyplot as plt
36
37     grouped = defaultdict(list)
38     for row in rows:
39         grouped[(row["matrix"], row["method"])] .append(row)
40
41     matrices = sorted({row["matrix"] for row in rows})
42     fig, axes = plt.subplots(len(matrices), 1, figsize=(9, 3.2 * len(
43         matrices))), squeeze=False)
44
45     for ax, matrix_name in zip(axes[:, 0], matrices):
46         for (current_matrix, method), values in grouped.items():
47             if current_matrix != matrix_name:
48                 continue
49             values = sorted(values, key=lambda item: float(item["tol"
50 ]))

```

```

46         xs = [float(item["tol"]) for item in values]
47         ys = [float(item[metric]) for item in values]
48         ax.plot(xs, ys, marker="o", label=method)
49
50         ax.set_title(matrix_name)
51         ax.set_xlabel("Tolleranza")
52         ax.set_ylabel(ylabel)
53         ax.set_xscale("log")
54         ax.invert_xaxis()
55         if log_y:
56             ax.set_yscale("log")
57         ax.grid(True, which="both", alpha=0.3)
58         ax.legend()
59
60     fig.tight_layout()
61     fig.savefig(path, dpi=200)
62     plt.close(fig)

```

A.4 analyze_cond.py

```

1  """Stima del numero di condizionamento delle matrici di test.
2
3  NOTA: questo script serve SOLO all'analisi dei risultati nella
4  relazione
5  (spiegare perche' alcuni metodi sono piu' lenti). NON fa parte della
6  mini
7  libreria di solutori: qui usiamo scipy.sparse.linalg.eigsh, che e' un
8  calcolatore di autovalori, per stimare lambda_max e lambda_min e
9  quindi
10 cond(A) = lambda_max / lambda_min.
11
12 Uso:
13     python analyze_cond.py
14 """
15 from __future__ import annotations
16
17 from pathlib import Path
18
19 from scipy.io import mmread
20 from scipy.sparse.linalg import eigsh
21
22 BASE_DIR = Path(__file__).resolve().parent
23 DATA_DIR = BASE_DIR / "dati"
24
25 def main() -> None:
26     paths = sorted(DATA_DIR.glob("*.mtx"))
27     print(f"{'matrice':<12s}{'n':>7s}{'lambda_min':>14s}{'lambda_max':>14s}{'cond(A)':>14s}")
28     print("-" * 61)
29     for path in paths:
30         A = mmread(path).tocsr().astype(float)
31         n = A.shape[0]
32         lam_max = eigsh(A, k=1, which="LM", return_eigenvectors=False)[0]
33         # piu' piccolo autovalore via shift-invert attorno a 0 (A SPD => lambda_min > 0)

```

```

32     lam_min = float(eigsh(A, k=1, sigma=0.0, which="LM",
33                        return_eigenvectors=False)[0])
34     cond = lam_max / lam_min
35     print(f"{path.name:<12s}{n:>7d}{lam_min:>14.3e}{lam_max:>14.3
36           e}{cond:>14.3e}")
37
38 if __name__ == "__main__":
39     main()

```

A.5 test_assignment.py

```

1  """Small tests for the assignment code."""
2
3  from __future__ import annotations
4
5  from pathlib import Path
6  from tempfile import TemporaryDirectory
7
8  import numpy as np
9  from scipy.io import mmread
10 from scipy.sparse import diags
11
12 from solvers import SOLVERS, ConjugateGradient
13
14
15 def test_scipy_reads_matrix_market() -> None:
16     text = """%%MatrixMarket matrix coordinate real general
17 % tiny test matrix
18 3 3 5
19 1 1 2.0
20 1 2 -1.0
21 2 1 -1.0
22 2 2 2.0
23 3 3 3.0
24 """
25     with TemporaryDirectory() as tmp:
26         path = Path(tmp) / "tiny.mtx"
27         path.write_text(text, encoding="utf-8")
28         A = mmread(path).tocsr()
29
30         x = np.array([1.0, 2.0, 3.0])
31         assert np.allclose(A @ x, np.array([0.0, 3.0, 9.0]))
32         assert np.allclose(A.diagonal(), np.array([2.0, 2.0, 3.0]))
33
34
35 def test_solvers_on_tridiagonal_spd() -> None:
36     A = _tridiagonal_spd(20)
37     x_exact = np.ones(20)
38     b = A @ x_exact
39
40     for solver in SOLVERS:
41         result = solver.solve(A, b, tol=1e-8, max_iter=20000)
42         relative_error = np.linalg.norm(x_exact - result.x) / np.
43             linalg.norm(x_exact)
44         assert result.converged, solver.name
45         assert relative_error < 1e-6, solver.name

```

```

45
46
47 def test_conjugate_gradient_is_fast_on_small_matrix() -> None:
48     A = _tridiagonal_spd(20)
49     x_exact = np.ones(20)
50     b = A @ x_exact
51     result = ConjugateGradient().solve(A, b, tol=1e-10, max_iter
52                                         =20000)
53     assert result.converged
54     assert result.iterations <= 20
55
56 def _tridiagonal_spd(n: int):
57     return diags(
58         diagonals=[-np.ones(n - 1), 3 * np.ones(n), -np.ones(n - 1)],
59         offsets=[-1, 0, 1],
60         format="csr",
61     )
62
63
64 if __name__ == "__main__":
65     test_scipy_reads_matrix_market()
66     test_solvers_on_tridiagonal_spd()
67     test_conjugate_gradient_is_fast_on_small_matrix()
68     print("Tutti i test sono passati.")

```